



Merge K Sorted Lists (medium)

We'll cover the following ^

- Problem Statement
- Try it yourself
- Solution
 - Code
 - Time complexity
 - Space complexity

Problem Statement

Given an array of 'K' sorted LinkedLists, merge them into one sorted list.

Example 1:

Input: L1=[2, 6, 8], L2=[3, 6, 7], L3=[1, 3, 4]
Output: [1, 2, 3, 3, 4, 6, 6, 7, 8]

Example 2:

Input: L1=[5, 8, 9], L2=[1, 7]
Output: [1, 5, 7, 8, 9]

Try it yourself

Try solving this question here:

Java

Python3

JS

C++

```
1 import java.util.*;
2
3 class ListNode {
4     int value;
5     ListNode next;
6
7     ListNode(int value) {
8         this.value = value;
9     }
10 }
11
12 class MergeKSortedLists {
13
14     public static ListNode merge(ListNode[] lists) {
15         ListNode result = new ListNode(-1);
16         // TODO: Write your code here
```



```

16 // TODO: write your code here
17 return result;
18 }
19
20 public static void main(String[] args) {
21     ListNode l1 = new ListNode(2);
22     l1.next = new ListNode(6);
23     l1.next.next = new ListNode(8);
24
25     ListNode l2 = new ListNode(3);
26     l2.next = new ListNode(6);
27     l2.next.next = new ListNode(7);
28
29     ListNode l3 = new ListNode(1);
30     l3.next = new ListNode(3);
31     l3.next.next = new ListNode(4);
32
33     ListNode result = MergeSortedLists.merge(new ListNode[] { l1, l2, l3 });
34     System.out.print("Here are the elements form the merged list: ");
35     while (result != null) {
36         System.out.print(result.value + " ");
37         result = result.next;
38     }
39 }
40 }
41

```

Run

Save

Reset



Solution

A brute force solution could be to add all elements of the given 'K' lists to one list and sort it. If there are a total of 'N' elements in all the input lists, then the brute force solution will have a time complexity of $O(N * \log N)$ as we will need to sort the merged list. Can we do better than this? How can we utilize the fact that the input lists are individually sorted?

If we have to find the smallest element of all the input lists, we have to compare only the smallest (i.e. the first) element of all the lists. Once we have the smallest element, we can put it in the merged list. Following a similar pattern, we can then find the next smallest element of all the lists to add it to the merged list.

The best data structure that comes to mind to find the smallest number among a set of 'K' numbers is a **Heap**. Let's see how can we use a heap to find a better algorithm.

1. We can insert the first element of each array in a **Min Heap**.
2. After this, we can take out the smallest (top) element from the heap and add it to the merged list.
3. After removing the smallest element from the heap, we can insert the next element of the same list into the heap.
4. We can repeat steps 2 and 3 to populate the merged list in sorted order.

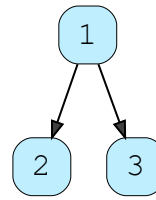
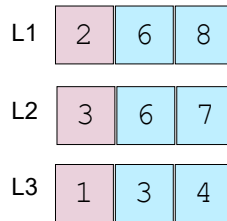
Let's take the Example-1 mentioned above to go through each step of our algorithm:

Given lists: $L1=[2, 6, 8]$, $L2=[3, 6, 7]$, $L3=[1, 3, 4]$

1. After inserting the 1st element of each list, the heap will have the following elements:



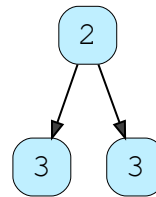
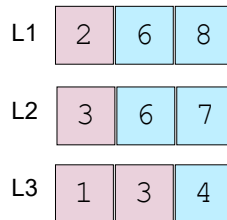
Given lists:



Insert the first number from each array in the heap

2. We'll take the top number from the heap, insert it into the merged list and add the next number in the heap.

Given lists:

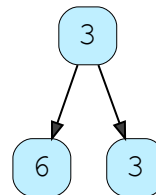
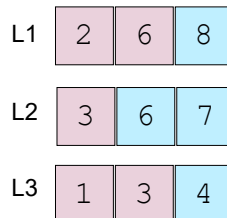


Merged List

1

3. Again, we'll take the top element of the heap, insert it into the merged list and add the next number to the heap.

Given lists:

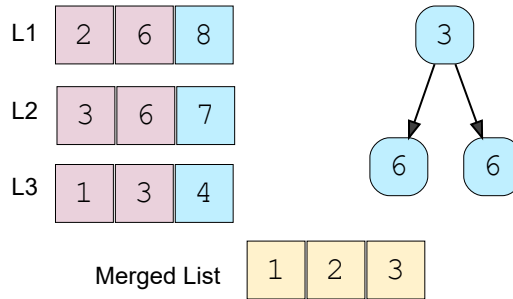


Merged List

1	2
---	---

4. Repeating the above step, take the top element of the heap, insert it into the merged list and add the next number to the heap. As there are two 3s in the heap, we can pick anyone but we need to take the next element from the corresponding list to insert in the heap.

Given lists:



We'll repeat the above step to populate our merged array.

Code

Here is what our algorithm will look like:



```

1  import java.util.*;
2
3  class ListNode {
4      int value;
5      ListNode next;
6
7      ListNode(int value) {
8          this.value = value;
9      }
10 }
11
12 class MergeKSortedLists {
13
14     public static ListNode merge(ListNode[] lists) {
15         PriorityQueue<ListNode> minHeap = new PriorityQueue<ListNode>((n1, n2) -> n1.value - n2.value);
16
17         // put the root of each list in the min heap
18         for (ListNode root : lists)
19             if (root != null)
20                 minHeap.add(root);
21
22         // take the smallest (top) element form the min-heap and add it to the result;
23         // if the top element has a next element add it to the heap
24         ListNode resultHead = null, resultTail = null;
25         while (!minHeap.isEmpty()) {
26             ListNode node = minHeap.poll();
27             if (resultHead == null) {
28                 resultHead = resultTail = node;
29             } else {
30                 resultTail.next = node;
31                 resultTail = resultTail.next;
32             }
33             if (node.next != null)
34                 minHeap.add(node.next);
35         }
36
37         return resultHead;
38     }
39
40     public static void main(String[] args) {

```

```
41     ListNode l1 = new ListNode(2);
42     l1.next = new ListNode(6);
43     l1.next.next = new ListNode(8);
44
45     ListNode l2 = new ListNode(3);
46     l2.next = new ListNode(6);
47     l2.next.next = new ListNode(7);
48
49     ListNode l3 = new ListNode(1);
50     l3.next = new ListNode(3);
51     l3.next.next = new ListNode(4);
52
53     ListNode result = MergeSortedList.merge(new ListNode[] { l1, l2, l3 });
54     System.out.print("Here are the elements form the merged list: ");
55     while (result != null) {
56         System.out.print(result.value + " ");
57         result = result.next;
58     }
59 }
60 }
61
```

[Run](#)[Save](#)[Reset](#)

Time complexity

Since we'll be going through all the elements of all arrays and will be removing/adding one element to the heap in each step, the time complexity of the above algorithm will be $O(N * \log K)$, where 'N' is the total number of elements in all the 'K' input arrays.

Space complexity

The space complexity will be $O(K)$ because, at any time, our min-heap will be storing one number from all the 'K' input arrays.

[← Back](#)☒ [Mark As Completed](#)[Next →](#)